Exploring Redpanda Console and VictoriaMetrics

A short guide to the 957 preview event log and the metrics UI. Console is one record per thing that happened. vmui is the counters and histograms. An auction id lives on the event. It is not a metric label.

Where to open them

UI	URL	What it shows
Redpanda Console	https://957.redpanda.bidon.squads.com/overview	Topics, the telemetry-events log, and the schema registry.
VictoriaMetrics (vmui)	https://957.metrics.bidon.squads.com/vmui	Rates, counts, and latencies scraped from sdkapi and the telemetry ingest.

The browser asks for basic auth. Console and vmui use different passwords. Ask a teammate for them. They are set in Coolify and are not in git. Scrape health for the same VictoriaMetrics is https://957.metrics.bidon.squads.com/targets.

The screenshots later in this guide were taken on a local cluster after the sample telemetry requests in sdk.http. The 957 preview runs the same Console (v2.8.15) and the same vmui. Topic size, message count, and timestamps there will be larger.

What you are looking at

sdkapi and the telemetry ingest both write protobuf records to the topic `telemetry-events`. VictoriaMetrics scrapes `/metrics` on sdkapi and on the ingest service every 15 seconds and keeps 7 days. A counter moves on the process immediately. vmui shows it on the next scrape.

Topic	Payload	When it has rows
`telemetry-events`	Protobuf. One message type per event.	After an auction, a show, a notice, or POST /v2/telemetry.
`notification-events`	JSON. One row per notice HTTP call.	After a win, loss, billing, or timeout notice.
`ad-events`	JSON. The older auction log, including device ids and raw bid bodies.	Off on this preview. KAFKA_PRODUCE_AD_EVENTS is false, so this topic is often missing.

Open `telemetry-events`. A show still writes a protobuf `ad_impression` and, for the notice HTTP call, a JSON row on `notification-events`.

Console, screen by screen

Start at https://957.redpanda.bidon.squads.com/overview. The left rail is Cluster, then Topics, then Schema Registry.

Cluster overview

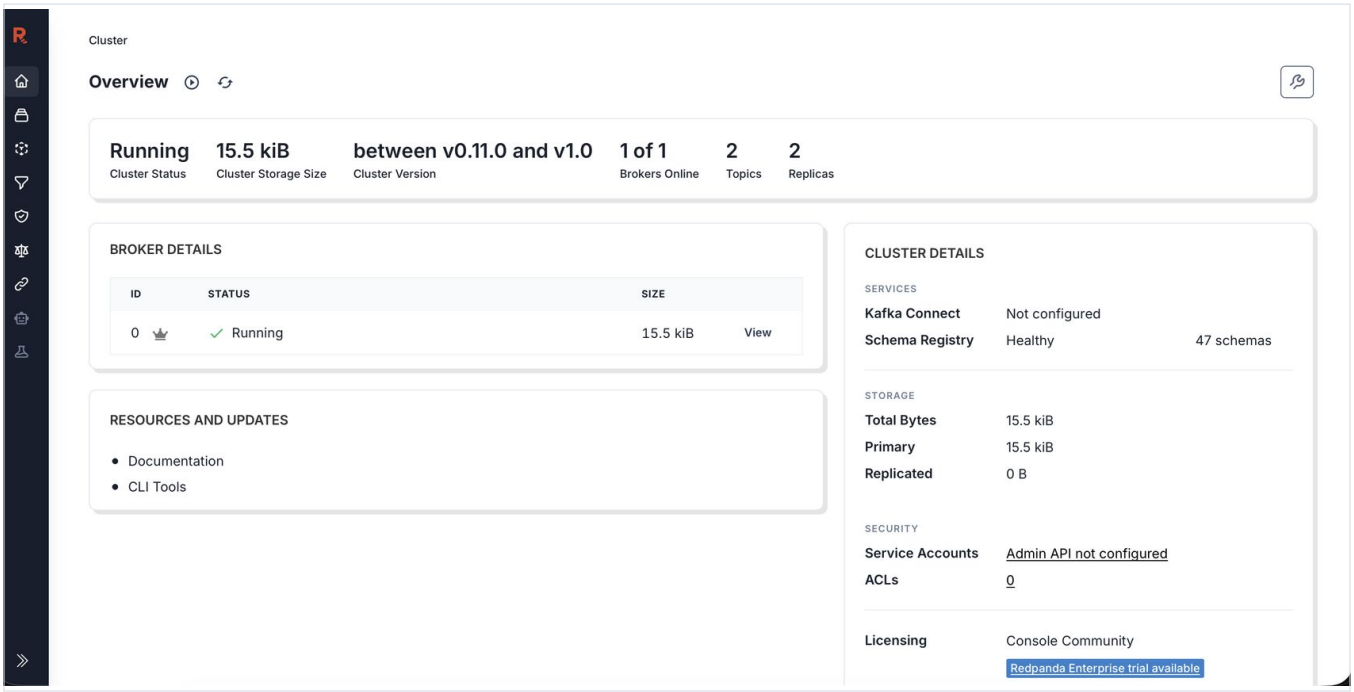


Figure 1. Cluster overview. Broker healthy, schema registry healthy, two topics.

Running and broker 0 healthy means Redpanda is up. The storage figure on a quiet local cluster is a few kilobytes. The 957 preview will be larger.

2 topics on that capture is `telemetry-events` plus Redpanda's internal `_schemas` topic. The Topics page hides internal topics until **Show internal topics** is checked, which is why the next figure lists one topic. **Kafka Connect: Not configured** is expected. Nothing in this stack sinks the topic. **Schema Registry: Healthy** and a schema count means the catalog subjects are registered. ACLs at 0 is this environment. The version line **between v0.11.0 and v1.0** is Console failing to map the broker's API versions onto one version string.

Topics

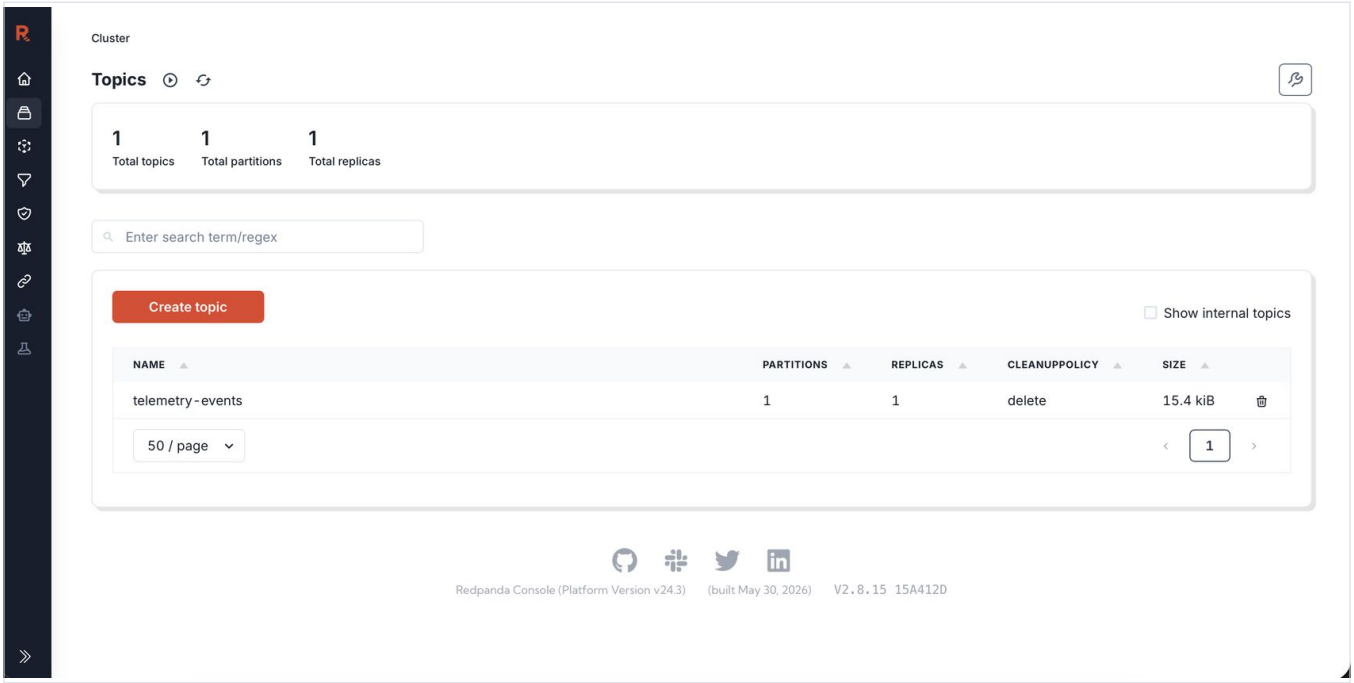


Figure 2. Topics. telemetry-events is the user topic. One partition, cleanup policy delete.

Click the topic name. Leave **Produce record** and **Delete records** alone. Those write or wipe the log. `notification-events` appears after the first notice. `ad-events` stays absent while produce is off.

Messages

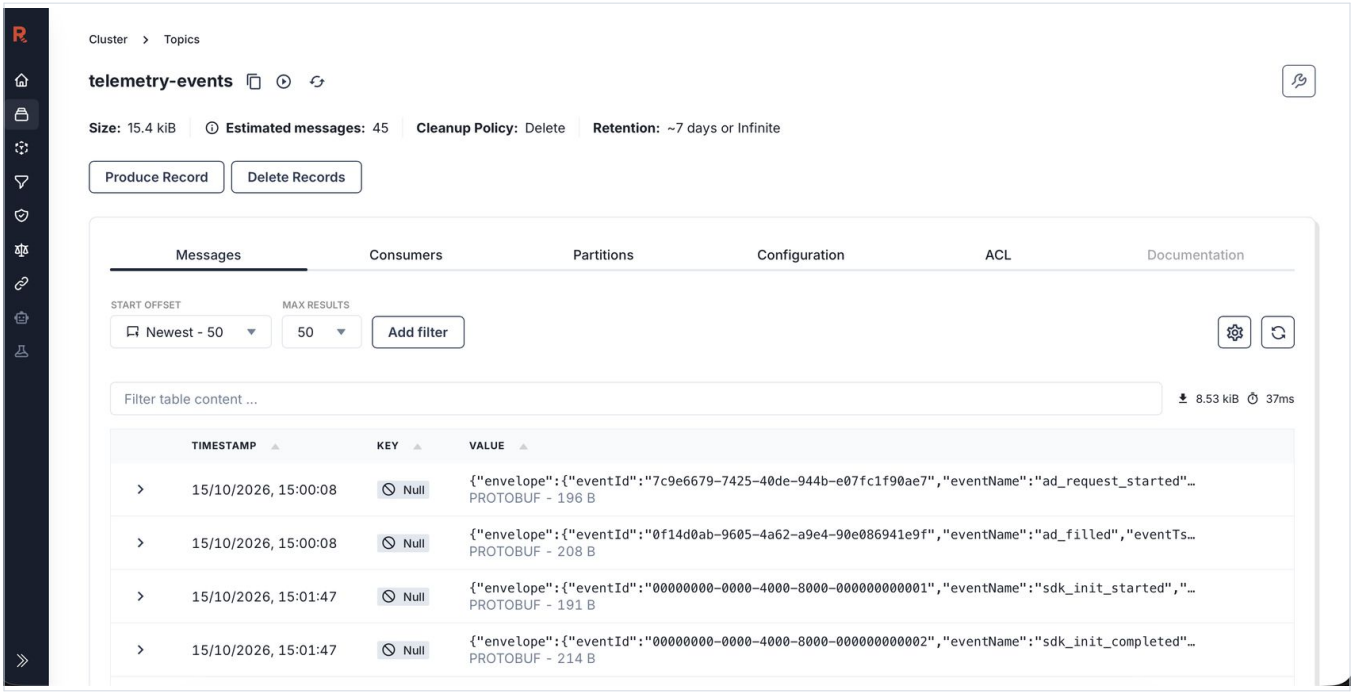


Figure 3. telemetry-events, newest 50. Protobuf previews for the sample client batches.

The page header shows size, an estimated message count, cleanup policy, and retention. On the local capture retention reads **~7 days or infinite** because no explicit retention was set. **Start offset: Newest - 50** and **max results 50** is the latest page. **Add filter** is a JavaScript filter that runs while reading the topic. The box under it only filters rows already on screen.

Column	How to read it
Timestamp	When the record was produced. This is not envelope.eventTs.
Key	Null. Telemetry records are not keyed.
Value	One-line preview of the decoded protobuf. eventName in that preview is the catalog event. PROTOBUF and a byte size means Console decoded it. A binary blob means the Confluent frame was missing.

The four rows in the figure are `ad_request_started`, `ad_filled` (15:00:08, from valid_request.json) and `sdk_init_started`, `sdk_init_completed` (15:01:47, from the catalogue fixture, event ids ending 0001 and 0002). The rest of that catalogue is further down the page.

valid_request.json also sends a `client_auction_duration_seconds` sample. It does not appear in this table. Metric samples are applied to /metrics and scraped into vmui. Only the events array becomes topic records.

One expanded record

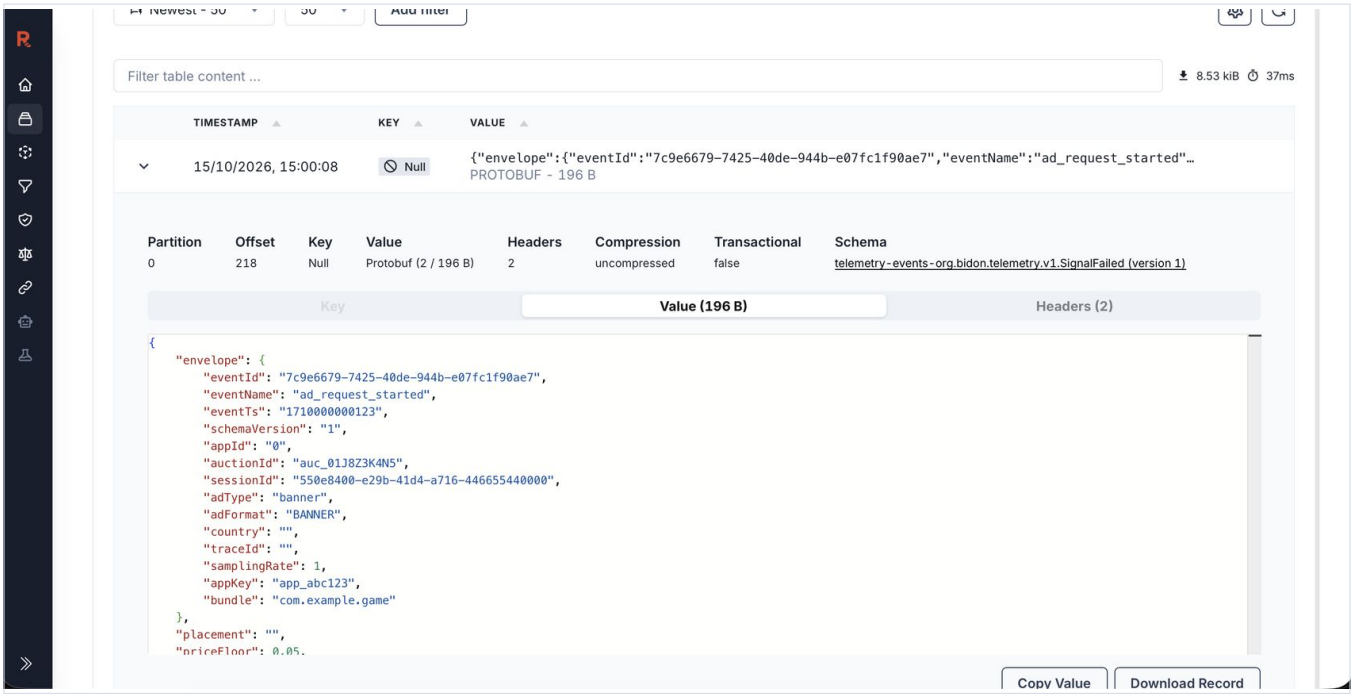


Figure 4. ad_request_started expanded. CamelCase JSON, two headers, schema link.

The chevron opens the record: partition, offset, key, size, header count, compression, and a schema link. **Value** is protobuf rendered as JSON with camelCase names. int64 fields are JSON strings. int32 fields are numbers.

JSON field	On this record
`eventName`	ad_request_started. This is the event type.
`eventTs`	"1710000000123", the sample clock: 9 Mar 2024 16:00:00.123 UTC. The table timestamp is the day the fixture was posted.
`schemaVersion`	1. The current catalog major.
`appId`	"0". Client ingest does not know the numeric app id. sdkapi events fill the real id.
`appKey, bundle`	From the batch app object: app_abc123, com.example.game.
`auctionId, sessionId, adType, adFormat`	Copied from the event. sessionId falls back to the batch session.
`country, traceId`	Empty. Ingest clears both. Local and current server auctions also leave traceId empty.
`samplingRate`	1. Nothing is sampled.
`placement`	"". Proto3 turns an unset string into an empty string. priceFloor further down is 0.05, which the sample did send.

Headers (2) are `event_name` and `protobuf_message`. Open that tab when the value did not decode. The header still names the event.

The **Schema** link on this row says `telemetry-events-org.bidon.telemetry.v1.SignalFailed` while the body is `ad_request_started`. Every subject is the same events.proto file, so the registry stores them under one schema id. Console looks that id up and prints one subject name for it. The message index inside the record selects ad_request_started, which is why placement and priceFloor are in the JSON. SignalFailed has neither of those fields. Read envelope.eventName. The link still opens the shared proto, which contains every message.

Schema Registry

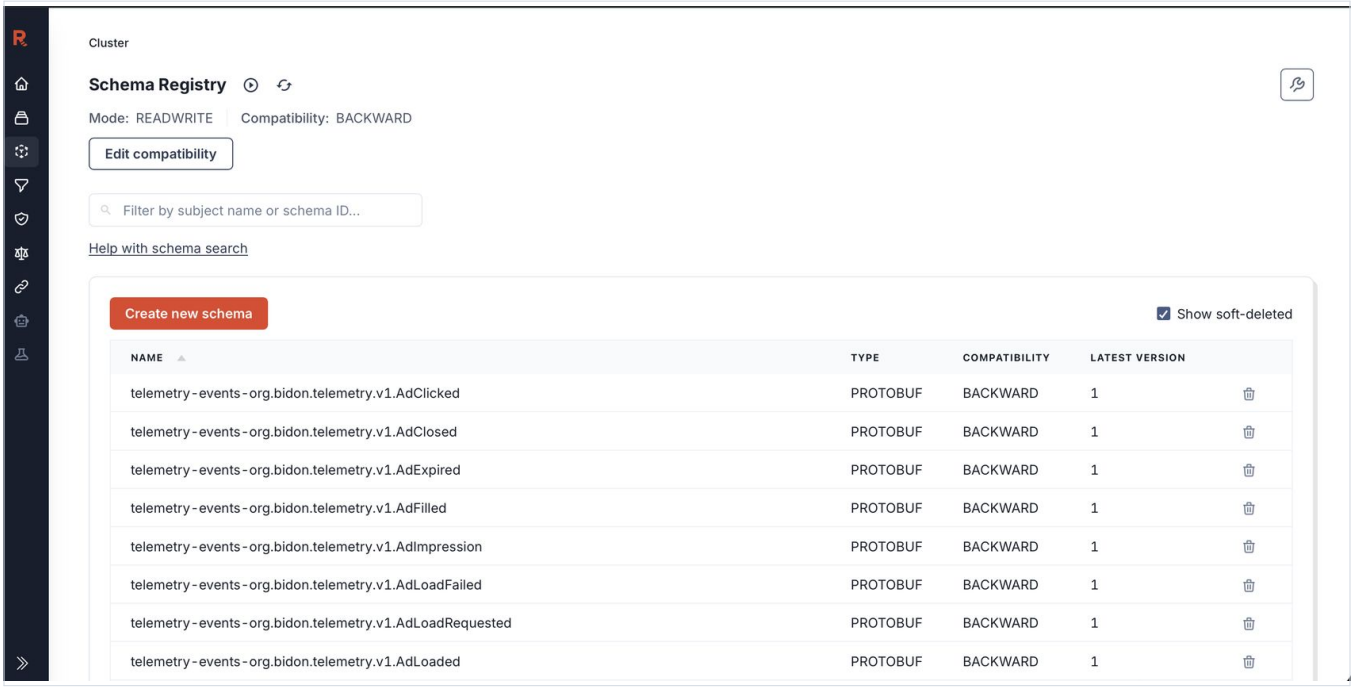


Figure 5. Schema Registry. One PROTOBUF subject per message type, compatibility BACKWARD, version 1.

Mode **READWRITE**, default compatibility **BACKWARD**. Each row is one message type, named `telemetry-events-org.bidon.telemetry.v1.<Message>`, type PROTOBUF, latest version 1. The list is alphabetical, so this shot is the Ad* types. Filter the box for AuctionCompleted or DspResponseReceived. Those subjects appear once sdkapi has registered them. Version 1 means the proto file has not been registered again since the first successful write.

Filtering the log

On the Messages page choose **Add filter**, then **JavaScript Filter**. Return true to keep the record. Headers arrive as bytes. This still works when the value is raw protobuf.

```
function header(name) {
  const raw = headers[name];
  if (!raw) return "";
  return String.fromCharCode(...new Uint8Array(raw));
}
return header("event_name") === "dsp_response_received";
```

One auction, once the value has decoded:

```
return value.envelope
  && value.envelope.auctionId === "PASTE_AUCTION_ID";
```

A DSP timeout:

```
return value.outcome === "OUTCOME_TIMEOUT";
```

A below-floor reject:

```
return value.rejectReason === "REJECT_REASON_BELOW_FLOOR";
```

On notification-events the payload is already JSON, so names stay snake_case.

```
return value.event_type === "NURL";
```

Server events on telemetry-events

sdkapi writes these. POST /v2/telemetry refuses the five auction and DSP names. It accepts the impression and notice names. The rows after a real auction come from sdkapi.

Bidding round

One auction writes the request, then one send and one response per DSP that was actually called, then the completion. participantCount on the completion equals that DSP count. Amazon is one response per request. A below-floor bid adds dsp_response_rejected.

event_name	When	Fields past the envelope
`auction_request_received`	Auction accepted, before the effective floor replaces the SDK floor.	requestedPriceFloor, the floor the SDK sent.
`dsp_request_sent`	Immediately before the adapter HTTP call.	dsp, scope (SCOPE_BIDDING_ROUND), timeoutBudgetMs (0 for Amazon).
`dsp_response_received`	Adapter returned, once per send.	dsp, outcome, httpStatus, latencyMs (send to return), price when it bid, nbr, creativeId, crtype.
`dsp_response_rejected`	A bid existed and price did not beat the effective floor.	dsp, price, priceFloor, rejectReason. Only REJECT_REASON_BELOW_FLOOR today.
`auction_completed`	Round finished, including failures.	winnerDsp, price, priceFloor (effective floor), participantCount, totalLatencyMs, errorCode.

outcome in the JSON	What happened
`OUTCOME_TIMEOUT`	The call hit its deadline.
`OUTCOME_HTTP_ERROR`	HTTP 4xx or 5xx, or a transport error with status 0.
`OUTCOME_MALFORMED`	A body came back and could not be parsed.
`OUTCOME_BID`	A bid.
`OUTCOME_NOBID`	A clean no-bid, such as 204 or an empty seat.

Timeout is chosen before the HTTP and parse checks. errorCode on auction_completed is ERROR_CODE_NO_ADS_FOUND, ERROR_CODE_INVALID_AUCTION_KEY, or ERROR_CODE_ERROR. Empty means the round finished without an auction error. winnerDsp is empty when nothing cleared the effective floor. A round with no winner and no error is still a successful completion.

Show and notices

POST /v2/show writes ad_impression. If that request already carried a client ad_impression with an eventId, that row is published as-is and sdkapi does not write a second one. Otherwise sdkapi synthesizes one with dsp and price. renderDurationMs and omEnabled stay at zero.

event_name	When	Fields past the envelope
`ad_impression`	Show.	dsp, price. A client row may also have renderDurationMs and omEnabled.
`win_notice_sent`	nurl attempt, after retries.	dsp, clearingPrice, success, httpStatus, retryCount, latencyMs.
`billing_notice_sent`	burl attempt.	dsp, billablePrice, success, httpStatus, retryCount, latencyMs.
`loss_notice_sent`	lurl attempt, and also a TimeoutURL.	dsp, lossReasonCode, winningPrice, success, httpStatus, retryCount, latencyMs.
`notice_delivery_failed`	The same attempt, only when success is false.	dsp, noticeType, httpStatus, attempts, latencyMs, errorCode.

noticeType is NOTICE_TYPE_WIN, NOTICE_TYPE_BILLING, NOTICE_TYPE_LOSS, or NOTICE_TYPE_TIMEOUT. A TimeoutURL is stored as loss_notice_sent with NOTICE_TYPE_TIMEOUT. errorCode on the failure event is http_4xx, http_5xx, or transport. HTTP 2xx succeeds. 4xx is terminal. 5xx is retried, then recorded with success false.

notification-events is the same notice call as JSON. event_type is NURL, BURL, LURL, or TimeoutURL. Useful fields: auction_id, demand_id, ecpm, url, error, loss_reason. The protobuf row adds status, retries, and latency.

Client events on the same topic

POST /v2/telemetry publishes every other catalog name onto telemetry-events. Filter with the same event_name header. The envelope has appKey and bundle, appId 0, and an empty traceId. A batch that names a server auction event, or a metric that is not allow-listed, is dropped and counted on ingest_total. Fields missing from the proto are dropped without a counter. The event is still accepted and the HTTP response is still 202.

Area	event_name
Init	sdk_init_started, sdk_init_completed, sdk_init_failed, adapter_init_result, config_fetch_result
Tokens	adapter_token_requested, adapter_token_result, token_collection_completed
Client auction	ad_request_started, auction_requested, auction_response_received, ad_filled, auction_failed, auction_no_demand
Load	ad_load_requested, adm_parse_result, renderer_selected, ad_loaded, ad_load_failed, ad_expired
Show	ad_show_requested, ad_impression, ad_show_failed, ad_viewable, ad_clicked, ad_closed, ad_reward_granted
Render	webview_error, render_crashed, sdk_crashed
Video	video_start, video_q1, video_midpoint, video_q3, video_complete, video_skipped, video_error
MAX adapter	adapter_load_budget_exceeded, no_fill_returned_to_max, signal_requested, signal_provided, signal_failed, signal_timeout_suspected

sdk.http has a request that posts one of each client event. After it, filter telemetry-events on any name in the table.

VictoriaMetrics

Open https://957.metrics.bidon.squads.com/vmui. Use the **Query** tab. Set the time range to the last 15 or 30 minutes, run the query, and switch between table and graph. **Explore**, then **Explore cardinality**, lists metric names and which labels they have.

job	Process	What is there
`bidon-sdkapi`	sdkapi	Auction, DSP, notice, and impression metrics, plus ordinary HTTP metrics.
`bidon-telemetry`	ingest on :1326	Client ingest results and the three client histograms.

Labels stay low-cardinality on purpose. There is no auction_id, app_id, or event_id. To see one auction, use Console. vmui answers how many and how slow across the window.

Counters only go up. `increase(...[15m])` is the count over the window. `rate(...[5m])` is per second. A histogram query uses the `_bucket` series. Series appear after the first event of that kind and the next scrape. `sdkapi_requests_total` is there as soon as sdkapi has served traffic.

Is the scrape alive?

```
up
```

```
sum by (url, code) (increase(sdkapi_requests_total[15m]))
```

Auctions

result is ok or error. error means the auction Run returned an error. A round that finishes with no winner and no error is still ok. The empty winnerDsp is on the event, not on this counter.

```
sum by (result) (increase(auction_completed_total[15m]))
```

DSP outcomes

outcome is the short label: bid, nobid, timeout, http_error, malformed. Same decision as OUTCOME_* on dsp_response_received, without the prefix.

```
sum by (dsp, outcome) (increase(dsp_response_total[15m]))
```

Timeouts for one DSP, as a share of its responses:

```
sum by (dsp) (rate(dsp_response_total{outcome="timeout"}[5m]))
/
sum by (dsp) (rate(dsp_response_total[5m]))
```

DSP latency

dsp_request_duration_seconds is send-to-return, labeled only by dsp. Buckets are 50ms, 100ms, 250ms, 500ms, 1s, 2s, 4s, and 8s. A latency above 8 seconds sits in the +Inf bucket. Swap 0.95 for 0.50 for a median. The graph stays empty until that DSP has a few samples inside the window.

```
histogram_quantile(
  0.95,
  sum by (le, dsp) (
    rate(dsp_request_duration_seconds_bucket[5m])
  )
)
```

Dropped telemetry

This should stay flat. A non-zero reason means a catalog event never reached telemetry-events.

```
sum by (reason) (increase(telemetry_dropped_total[15m]))
```

reason	Meaning
`invalid`	Missing event name or envelope.
`marshal`	Protobuf encode failed.
`buffer_full`	The producer buffer was full. Counted, and not logged on every row.
`produce_error`	The broker rejected the record.

Notices and impressions

type is nurl, burl, lurl, or timeout (a TimeoutURL). result is 2xx, 4xx, 5xx, or timeout (a transport failure, status 0). The word timeout means a TimeoutURL when it is the type, and a failed connection when it is the result.

```
sum by (type, result) (increase(notice_delivery_total[15m]))
```

```
increase(ad_impression_total[15m])
```

ad_impression_total counts rows emitted, synthesized or client. It has no labels.

Client ingest

These series are on job bidon-telemetry. They stay absent until something calls POST /v2/telemetry.

```
sum by (result) (increase(ingest_total[15m]))
```

result	Meaning
`accepted`	Event produced, or a metric sample applied.
`dropped_name`	Unknown event_name, or the JSON did not match that message. Includes the five server auction and DSP names.
`dropped_metric`	Metric name not allow-listed, not a histogram, or an unexpected label key.
`buffer_full`	The ingest queue was full.
`produce_error`	Produce failed.

The only client histograms, and the only labels they accept:

Metric	Labels
client_auction_duration_seconds	ad_type
client_token_collection_duration_seconds	ad_type
client_ad_load_duration_seconds	ad_type, result

```
histogram_quantile(
  0.95,
  sum by (le, ad_type) (
    rate(client_auction_duration_seconds_bucket[15m])
  )
)
```

```
histogram_quantile(
  0.95,
  sum by (le, ad_type, result) (
    rate(client_ad_load_duration_seconds_bucket[15m])
  )
)
```

Anything else in the batch metrics array increments dropped_metric and does not create a series.

One auction, both UIs

- Send an auction to the 957 preview sdkapi, or run one from the app.
- In Console, open telemetry-events from a recent offset and read envelope.auctionId off auction_request_received.
- Filter on that auctionId. You should see the request, a send and a response per DSP, any below-floor reject, and auction_completed. latencyMs on each response is that DSP only. totalLatencyMs on the completion is the whole round.
- Wait for a scrape, then in vmui run sum by (dsp, outcome) (increase(dsp_response_total[15m])). The outcomes match the outcome fields you just read, summed across every auction in the window.
- Call show. Console gains ad_impression, then billing_notice_sent and the other notice rows. notification-events gains a JSON row per notice with the same auction_id. notice_delivery_total and ad_impression_total move on the next scrape.

If Console has the events and vmui does not, open https://957.metrics.bidon.squads.com/targets. Both bidon-sdkapi and bidon-telemetry should be up. If the targets are up and the time range covers the auction, give the 15 second scrape a moment and run the query again.